VIDHYADEEP UNIVERSITY
Holy Flame Of Knowledge

Vidhyadeep Institute of Commerce & Information Technology, Anita (Kim)

PRACTICAL RECORD

# UNIX AND SHELL PROGRAMMING

TY B.Sc. IT | Semester V

| STUDENT NAME | SHIVAM BHADORIYA | ENROLLMENT NO. | 240606302109 |
|---|---|---|---|

## INDEX


| NO. | PROGRAM / PRACTICAL | PAGE | SIGN |
|---|---|---|---|
| **ASSIGNMENT 2 - SHELL PROGRAMMING** | | | |
| 01 | Arithmetic operations | 2 | |
| 02 | Largest among three numbers | 2 | |
| 03 | Even/odd and sign | 3 | |
| 04 | Multiplication table | 3 | |
| 05 | Factorial | 3 | |
| 06 | Prime number test | 4 | |
| 07 | Prime numbers from 1 to 100 | 4 | |
| 08 | Fibonacci series | 4 | |
| 09 | Palindrome number | 5 | |
| 10 | Reverse a number | 5 | |
| 11 | Sum of digits | 6 | |
| 12 | Menu-driven calculator | 6 | |
| 13 | String operations | 7 | |
| 14 | File handling | 7 | |
| 15 | User-defined function | 8 | |
| **ASSIGNMENT 3 - FILE PROCESSING UTILITIES** | | | |
| 01 | Splitting files | 8 | |
| 02 | Comparing files | 9 | |
| 03 | Sorting student names and marks | 10 | |
| 04 | Merging files | 10 | |
| 05 | Ordering records by a field | 11 | |
| 06 | grep utility | 11 | |
| 07 | sed utility | 12 | |
| 08 | awk field processing | 12 | |
| 09 | awk calculations | 13 | |
| 10 | Combined file-utility practical | 13 | |

ASSIGNMENT 2 - SHELL PROGRAMMING

# UNIX & Shell Programming

## 01. Arithmetic operations

Aim: Write a shell script that accepts two numbers and performs addition, subtraction, multiplication, division, and modulus.

Logic: Read signed integers and use Bash arithmetic; reject division/modulus by zero.

**PROGRAM / q01.sh**

```
#!/usr/bin/env bash
read -r -p "Enter two integers: " a b
if ! [[ $a =~ ^-?[0-9]+$ && $b =~ ^-?[0-9]+$ ]]; then
  echo "Please enter valid integers."; exit 1
fi
echo "Addition: $((a + b))"
echo "Subtraction: $((a - b))"
echo "Multiplication: $((a * b))"
if ((b == 0)); then
  echo "Division and modulus are undefined for zero divisor."
else
  echo "Division: $((a / b))"
  echo "Modulus: $((a % b))"
fi
```

**OUTPUT**

```
Addition: 12
Subtraction: 8
Multiplication: 20
Division: 5
Modulus: 0
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

Note: Uses integer (truncated) division, consistent with Bash arithmetic.

## 02. Largest among three numbers

Aim: Accept three numbers and find the largest using if-elif-else.

Logic: Compare the three supplied integers, allowing ties.

**PROGRAM / q02.sh**

```
#!/usr/bin/env bash
read -r -p "Enter three integers: " a b c
for n in "$a" "$b" "$c"; do
  [[ $n =~ ^-?[0-9]+$ ]] || { echo "Invalid integer."; exit 1; }
done
if ((a >= b && a >= c)); then
  largest=$a
elif ((b >= a && b >= c)); then
  largest=$b
else
  largest=$c
fi
echo "Largest: $largest"
```

**OUTPUT**

```
Largest: 5
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

## 03. Even/odd and sign

Aim: Accept a number and report even/odd and positive/negative/zero.

Logic: Use remainder for parity and arithmetic comparisons for the sign.

**PROGRAM / q03.sh**

```
#!/usr/bin/env bash
read -r -p "Enter an integer: " n
[[ $n =~ ^-?[0-9]+$ ]] || { echo "Invalid integer."; exit 1; }
if ((n % 2 == 0)); then echo "Even"; else echo "Odd"; fi
if ((n > 0)); then
  echo "Positive"
elif ((n < 0)); then
  echo "Negative"
else
  echo "Zero"
fi
```

**OUTPUT**

```
Even
Positive
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

Note: Zero is even and neither positive nor negative.

---

## 04. Multiplication table

Aim: Accept a number and display its multiplication table from 1 to 10 using a loop.

Logic: Use a for loop and integer arithmetic.

**PROGRAM / q04.sh**

```
#!/usr/bin/env bash
read -r -p "Enter an integer: " n
[[ $n =~ ^-?[0-9]+$ ]] || { echo "Invalid integer."; exit 1; }
for ((i = 1; i <= 10; i++)); do
  printf '%d x %d = %d\n' "$n" "$i" "$((n * i))"
done
```

**OUTPUT**

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

---

## 05. Factorial

Aim: Compute the factorial of a non-negative number using a loop.

Logic: Start from one; explicitly reject negative input.

**PROGRAM / q05.sh**

```
#!/usr/bin/env bash
read -r -p "Enter a non-negative integer: " n
[[ $n =~ ^[0-9]+$ ]] || { echo "Invalid non-negative integer."; exit 1; }
result=1
for ((i = 2; i <= n; i++)); do ((result *= i)); done
printf '%d! = %d\n' "$n" "$result"
```

**OUTPUT**

```
5! = 120
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

Note: Large factorials overflow Bash signed integer arithmetic; use a big-integer tool for large n.

## 06. Prime number test

Aim: Check whether a given number is prime.

Logic: Reject n less than two; test divisors only up to the square root.

**PROGRAM / q06.sh**

```
#!/usr/bin/env bash
read -r -p "Enter an integer: " n
[[ $n =~ ^-?[0-9]+$ ]] || { echo "Invalid integer."; exit 1; }
if ((n < 2)); then echo "$n is not prime"; exit 0; fi
for ((i = 2; i * i <= n; i++)); do
  if ((n % i == 0)); then echo "$n is not prime"; exit 0; fi
done
echo "$n is prime"
```

**OUTPUT**

```
29 is prime
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

## 07. Prime numbers from 1 to 100

Aim: Display every prime between 1 and 100.

Logic: Use a nested loop and stop at the first discovered divisor.

**PROGRAM / q07.sh**

```
#!/usr/bin/env bash
for ((n = 2; n <= 100; n++)); do
  prime=1
  for ((i = 2; i * i <= n; i++)); do
    if ((n % i == 0)); then prime=0; break; fi
  done
  if ((prime)); then printf '%d ' "$n"; fi
done
printf '\n'
```

**OUTPUT**

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

## 08. Fibonacci series

Aim: Generate the first N terms of the Fibonacci series.

Logic: Print the current term, then update the pair for the next iteration.

**PROGRAM / q08.sh**

```
#!/usr/bin/env bash
read -r -p "How many terms? " n
[[ $n =~ ^[0-9]+$ ]] || { echo "Invalid term count."; exit 1; }
a=0; b=1
for ((i = 0; i < n; i++)); do
  printf '%d ' "$a"
  next=$((a + b)); a=$b; b=$next
done
printf '\n'
```

**OUTPUT**

```
0 1 1 2 3 5 8
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

Note: For long sequences, native Bash integers will eventually overflow.

## 09. Palindrome number

Aim: Accept a non-negative integer and check whether it reads the same backward.

Logic: Reverse digits with modulo and compare to the original.

**PROGRAM / q09.sh**

```
#!/usr/bin/env bash
read -r -p "Enter a non-negative integer: " n
[[ $n =~ ^[0-9]+$ ]] || { echo "Invalid integer."; exit 1; }
original=$n; reverse=0; temp=$n
while ((temp > 0)); do
  reverse=$((reverse * 10 + temp % 10))
  temp=$((temp / 10))
done
if ((original == reverse)); then
  echo "$original is a palindrome"
else
  echo "$original is not a palindrome"
fi
```

**OUTPUT**

```
12321 is a palindrome
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

Note: Integer interpretation does not preserve leading zeroes.

## 10. Reverse a number

Aim: Accept a number and display its reverse.

Logic: Accumulate decimal digits in reverse order.

**PROGRAM / q10.sh**

```
#!/usr/bin/env bash
read -r -p "Enter a non-negative integer: " n
[[ $n =~ ^[0-9]+$ ]] || { echo "Invalid integer."; exit 1; }
reverse=0
while ((n > 0)); do
  reverse=$((reverse * 10 + n % 10))
  n=$((n / 10))
done
echo "Reversed number: $reverse"
```

**OUTPUT**

```
Reversed number: 4321
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

Note: Leading zeroes of the reversed numeral are naturally omitted.

## 11. Sum of digits

Aim: Accept a number and calculate the sum of its digits.

Logic: Repeatedly take the last digit with % 10, then divide by ten.

**PROGRAM / q11.sh**

```
#!/usr/bin/env bash
read -r -p "Enter a non-negative integer: " n
[[ $n =~ ^[0-9]+$ ]] || { echo "Invalid integer."; exit 1; }
sum=0
while ((n > 0)); do
  sum=$((sum + n % 10))
  n=$((n / 10))
done
echo "Sum of digits: $sum"
```

**OUTPUT**

```
Sum of digits: 6
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

## 12. Menu-driven calculator

Aim: Create a case-based menu for addition, subtraction, multiplication, division, modulus, and exit.

Logic: Validate the menu selection and operands; repeat until option six is selected.

**PROGRAM / q12.sh**

```
#!/usr/bin/env bash
while true; do
  printf '\n1. Add  2. Subtract  3. Multiply  4. Divide\n'
  printf '5. Modulus  6. Exit\n'
  read -r -p "Choose 1-6: " choice
  case "$choice" in
    6) echo "Goodbye"; break ;;
    1|2|3|4|5) ;;
    *) echo "Invalid choice"; continue ;;
  esac
  read -r -p "Enter two integers: " a b
  if ! [[ $a =~ ^-?[0-9]+$ && $b =~ ^-?[0-9]+$ ]]; then
    echo "Invalid integer input"; continue
  fi
  if [[ $choice == 4 || $choice == 5 ]] && ((b == 0)); then
    echo "Cannot use zero as divisor"; continue
  fi
  case "$choice" in
    1) answer=$((a+b)) ;; 2) answer=$((a-b)) ;;
    3) answer=$((a*b)) ;; 4) answer=$((a/b)) ;;
    5) answer=$((a%b)) ;;
  esac
  echo "Result: $answer"
done
```

**OUTPUT**

```


1. Add  2. Subtract  3. Multiply  4. Divide
5. Modulus  6. Exit
Result: 540

1. Add  2. Subtract  3. Multiply  4. Divide
5. Modulus  6. Exit
Goodbye
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

## 13. String operations

Aim: Find length, compare two strings, detect empty input, and reverse the first string.

Logic: Bash length and substring slicing are sufficient for a classroom illustration.

**PROGRAM / q13.sh**

```
#!/usr/bin/env bash
read -r -p "Enter a string: " first
read -r -p "Enter a second string: " second
printf 'Length: %d\n' "${#first}"
if [[ $first == "$second" ]]; then
  echo "Strings are equal"
else
  echo "Strings are different"
fi
if [[ -z $first ]]; then echo "First string is empty";
else echo "First string is not empty"; fi
reverse=''
for ((i=${#first}-1; i>=0; i--)); do
  reverse+=${first:i:1}
done
printf 'Reverse: %s\n' "$reverse"
```

**OUTPUT**

```
Length: 4
Strings are different
First string is not empty
Reverse: diaz
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

Note: Unicode grapheme clusters may need a specialized text library.

---

## 14. File handling

Aim: Check whether a path exists, whether it is a regular file/directory, and read/write/execute permissions.

Logic: Use Bash file test operators -e, -f, -d, -r, -w, and -x.

**PROGRAM / q14.sh**

```
#!/usr/bin/env bash
read -r -p "Enter path: " path
if [[ ! -e $path ]]; then echo "Path does not exist"; exit 0; fi
echo "Path exists"
if [[ -f $path ]]; then echo "Regular file";
elif [[ -d $path ]]; then echo "Directory";
else echo "Other file type"; fi
for mode in read write execute; do
  case "$mode" in
    read)    testflag=-r ;; write) testflag=-w ;;
    execute) testflag=-x ;;
  esac
  if test "$testflag" "$path"; then
    echo "$mode permission: yes"
  else
    echo "$mode permission: no"
  fi
done
```

**OUTPUT**

```
Path exists
Regular file
read permission: yes
write permission: yes
execute permission: no
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

Note: Permissions are evaluated for the user running the script; actual output depends on file mode.

---

## 15. User-defined function

Aim: Define and call a shell function to calculate factorial, sum of digits, or prime number.

Logic: Use a reusable factorial function, satisfying one permitted option.

**PROGRAM / q15.sh**

```
#!/usr/bin/env bash
factorial() {
  local n=$1 result=1 i
  if ! [[ $n =~ ^[0-9]+$ ]]; then
    echo "Factorial requires a non-negative integer"; return 1
  fi
  for ((i=2; i<=n; i++)); do ((result *= i)); done
  printf 'Factorial of %d = %d\n' "$n" "$result"
}
read -r -p "Enter a non-negative integer: " value
factorial "$value"
```

**OUTPUT**

```
Factorial of 10 = 3628800
```

*Output was captured from a real Bash execution in an isolated temporary directory.*

**ASSIGNMENT 3 - FILE PROCESSING UTILITIES**

# UNIX File Processing Utilities

## 01. Splitting files

Aim: Create a text file with at least twenty lines, split it into smaller files, and display the generated files.

Logic: Generate 20 numbered lines; split every five lines into deterministic part_ files.

**PROGRAM / c01.sh**

```
#!/usr/bin/env bash
for ((i=1; i<=20; i++)); do
  echo "This is line $i"
done > lines.txt
split -l 5 lines.txt part_
echo "Generated files:"
ls part_*
echo "All contents in order:"
cat part_*
```

**OUTPUT**

```
Generated files:
part_aa
part_ab
part_ac
part_ad
All contents in order:
This is line 1
This is line 2
This is line 3
This is line 4
This is line 5
This is line 6
This is line 7
This is line 8
This is line 9
This is line 10
This is line 11
This is line 12
This is line 13
This is line 14
This is line 15
This is line 16
This is line 17
This is line 18
This is line 19
This is line 20
```

*Output was captured from a real Bash execution in an isolated directory.*

## 02. Comparing files

Aim: Create two text files; compare them using cmp and diff, and explain the differences.

Logic: cmp locates the first differing byte; diff shows changed lines and additional lines.

**PROGRAM / c02.sh**

```
#!/usr/bin/env bash
cat > file1.txt <<'EOF'
Apple
Banana
Cherry
EOF
cat > file2.txt <<'EOF'
Apple
Blueberry
Cherry
Date
EOF
echo "First difference (cmp):"
cmp file1.txt file2.txt || true
echo "Changed lines (diff):"
diff file1.txt file2.txt || true
```

**OUTPUT**

```
First difference (cmp):
file1.txt file2.txt differ: char 8, line 2
Changed lines (diff):
2c2
< Banana
---
> Blueberry
3a4
> Date
```

*Output was captured from a real Bash execution in an isolated directory.*

Note: cmp returns status 1 for different files; the script deliberately proceeds to diff. diff notation such as 2c2 means line two changed.

## 03. Sorting student names and marks

Aim: Sort student names alphabetically, marks ascending, and marks descending with sort.

Logic: Preserve header separately so it does not participate in numeric sorting.

**PROGRAM / c03.sh**

```
#!/usr/bin/env bash
cat > students.txt <<'EOF'
Name|Marks
Riya|78
Aman|92
Neha|65
Karan|85
EOF
echo "Names A-Z:"
{ head -n 1 students.txt; tail -n +2 students.txt | sort -t'|' -k1,1; }
echo "Marks low-high:"
{ head -n 1 students.txt; tail -n +2 students.txt | sort -t'|' -k2,2n; }
echo "Marks high-low:"
{ head -n 1 students.txt; tail -n +2 students.txt | sort -t'|' -k2,2nr; }
```

**OUTPUT**

```
Names A-Z:
Name|Marks
Aman|92
Karan|85
Neha|65
Riya|78
Marks low-high:
Name|Marks
Neha|65
Riya|78
Karan|85
Aman|92
Marks high-low:
Name|Marks
Aman|92
Karan|85
Riya|78
Neha|65
```

*Output was captured from a real Bash execution in an isolated directory.*

## 04. Merging files

Aim: Create two text files and merge their contents into a third using UNIX commands.

Logic: Concatenate the files in the intended order with cat.

**PROGRAM / c04.sh**

```
#!/usr/bin/env bash
printf 'Apple\nBanana\n' > fruits1.txt
printf 'Cherry\nDate\n' > fruits2.txt
cat fruits1.txt fruits2.txt > merged_fruits.txt
echo "Merged file:"
cat merged_fruits.txt
```

**OUTPUT**

```
Merged file:
Apple
Banana
Cherry
Date
```

*Output was captured from a real Bash execution in an isolated directory.*

## 05. Ordering records by a field

Aim: Create student records and arrange them by name and marks using sort.

Logic: Use pipe as field delimiter: name is field two, marks are numeric field three.

**PROGRAM / c05.sh**

```
#!/usr/bin/env bash
cat > records.txt <<'EOF'
Roll|Name|Marks
101|Riya|78
102|Aman|92
103|Neha|65
104|Karan|85
EOF
echo "Sorted by name:"
{ head -n 1 records.txt; tail -n +2 records.txt | sort -t'|' -k2,2; }
echo "Sorted by marks:"
{ head -n 1 records.txt; tail -n +2 records.txt | sort -t'|' -k3,3n; }
```

**OUTPUT**

```
Sorted by name:
Roll|Name|Marks
102|Aman|92
104|Karan|85
103|Neha|65
101|Riya|78
Sorted by marks:
Roll|Name|Marks
103|Neha|65
101|Riya|78
104|Karan|85
102|Aman|92
```

*Output was captured from a real Bash execution in an isolated directory.*

---

## 06. grep utility

Aim: Search for a name and word, count matching lines, and search case-insensitively using grep.

Logic: Demonstrate matching output, count only, and the -i flag.

**PROGRAM / c06.sh**

```
#!/usr/bin/env bash
cat > student_info.txt <<'EOF'
Aman studies Computer Science
Riya studies Information Technology
Neha studies Computer Science
Karan studies Mathematics
EOF
echo "Name Riya:"
grep 'Riya' student_info.txt
echo "Containing Computer:"
grep 'Computer' student_info.txt
echo "Count containing Computer:"
grep -c 'Computer' student_info.txt
echo "Case-insensitive riya:"
grep -i 'riya' student_info.txt
```

**OUTPUT**

```
Name Riya:
Riya studies Information Technology
Containing Computer:
Aman studies Computer Science
Neha studies Computer Science
Count containing Computer:
2
Case-insensitive riya:
Riya studies Information Technology
```

*Output was captured from a real Bash execution in an isolated directory.*

## 07. sed utility

Aim: Use sed to print selected lines, substitute a word, delete a specified line, and display matching lines.

Logic: Use -n for selective printing; substitutions/deletions here do not modify the source file.

**PROGRAM / c07.sh**

```
#!/usr/bin/env bash
cat > messages.txt <<'EOF'
The red rose is beautiful.
The blue sky is clear.
The red car is fast.
The green tree is tall.
EOF
echo "Lines 2-3:"
sed -n '2,3p' messages.txt
echo "Replace red with yellow:"
sed 's/red/yellow/g' messages.txt
echo "Delete line 2:"
sed '2d' messages.txt
echo "Lines containing tree:"
sed -n '/tree/p' messages.txt
```

**OUTPUT**

```
Lines 2-3:
The blue sky is clear.
The red car is fast.
Replace red with yellow:
The yellow rose is beautiful.
The blue sky is clear.
The yellow car is fast.
The green tree is tall.
Delete line 2:
The red rose is beautiful.
The red car is fast.
The green tree is tall.
Lines containing tree:
The green tree is tall.
```

*Output was captured from a real Bash execution in an isolated directory.*

Note: The replacement and deletion are preview operations; the underlying messages.txt stays unchanged.

## 08. awk field processing

Aim: Create a student file (roll no., name, course, marks) and display selected fields with awk.

Logic: Use -F to set pipe as delimiter and print the name/course columns.

**PROGRAM / c08.sh**

```
#!/usr/bin/env bash
cat > student_details.txt <<'EOF'
Roll|Name|Course|Marks
101|Aman|BCA|92
102|Riya|BScIT|78
103|Neha|BCA|65
EOF
awk -F'|' 'NR==1 {print "Name    Course"; next}
{print $2 "    " $3}' student_details.txt
```

**OUTPUT**

```
Name    Course
Aman    BCA
Riya    BScIT
Neha    BCA
```

*Output was captured from a real Bash execution in an isolated directory.*

## 09. awk calculations

Aim: Create marks records and calculate total and average marks for each student using awk.

Logic: Treat fields two through four as numeric values and print average to two decimals.

**PROGRAM / c09.sh**

```
#!/usr/bin/env bash
cat > marks.txt <<'EOF'
Name|English|Maths|Computer
Aman|80|90|95
Riya|75|82|88
Neha|65|70|72
EOF
awk -F'|' 'NR==1 {print "Name    Total    Average"; next}
{total=$2+$3+$4; printf "%s    %d    %.2f\n", $1,total,total/3}' marks.txt
```

**OUTPUT**

```
Name    Total    Average
Aman    265    88.33
Riya    245    81.67
Neha    207    69.00
```

*Output was captured from a real Bash execution in an isolated directory.*

## 10. Combined file-utility practical

Aim: Use sort, grep, sed, and awk to sort salaries, filter department, rename department in output, and filter salaries above a limit.

Logic: Each utility operates on the same pipe-delimited employee dataset.

**PROGRAM / c10.sh**

```
#!/usr/bin/env bash
cat > employees.txt <<'EOF'
ID|Name|Department|Salary
E103|Neha|Sales|45000
E101|Aman|IT|60000
E104|Karan|Sales|52000
E102|Riya|HR|48000
EOF
echo "Sorted by salary:"
{ head -n 1 employees.txt; tail -n +2 employees.txt | sort -t'|' -k4,4n; }
echo "Department Sales:"
grep '|Sales|' employees.txt
echo "Sales renamed to Marketing in displayed text:"
sed 's/|Sales|/|Marketing|/g' employees.txt
echo "Salary above 50000:"
awk -F'|' 'NR==1 || $4>50000' employees.txt
```

**OUTPUT**

```
Sorted by salary:
ID|Name|Department|Salary
E103|Neha|Sales|45000
E102|Riya|HR|48000
E104|Karan|Sales|52000
E101|Aman|IT|60000
Department Sales:
E103|Neha|Sales|45000
E104|Karan|Sales|52000
Sales renamed to Marketing in displayed text:
ID|Name|Department|Salary
E103|Neha|Marketing|45000
E101|Aman|IT|60000
E104|Karan|Marketing|52000
E102|Riya|HR|48000
Salary above 50000:
ID|Name|Department|Salary
E101|Aman|IT|60000
E104|Karan|Sales|52000
```

*Output was captured from a real Bash execution in an isolated directory.*